Reinforcement Learning Lab

Lesson 6: TensorFlow-PyTorch and Neural Networks

Alberto Castellini, Riccardo Fidanza

University of Verona
email: alberto.castellini@univr.it, riccardo.fidanza@studenti.univr.it

Academic Year 2025-26

UNIVERSITÀ di VERONA
Dipartimento di INFORMATICA

Environment Setup

Safe Procedure

Always back up the previous lessons' solutions before executing the repository update.

The first step for the setup of the laboratory environment is to update the repository and load the miniconda environment.

- Update the repository of the lab:

```
cd RL-Lab
git stash
git pull
git stash pop
```

- Activate the *miniconda* environment:

```
conda activate rl-lab
```

- Install new dependencies

```
pip3 install torch torchvision torchaudio
#(if you are using Linux even with Cuda available, MacOS or Windows)


sudo apt install graphviz (optional)
pip install torchviz (optional)
```

We refer to the original PyTorch page https://pytorch.org for installation instruction.

Second Tutorial

The README file on the repository (link) contains a couple of tutorials on TensorFlow and PyTorch, useful to complete today's and next assignments, in particular:

Optimization

TensorFlow and PyTorch are packages for non-linear optimization problems. Typically used to create, train, and deploy artificial neural networks. In the first part of the tutorials, there are examples of non-linear problems solved with these packages.

Create a Neural Network

In the second part of the tutorials, we will learn how to create a deep neural network using both TensorFlow and PyTorch.

Train a Neural Network

Finally, in the last part of the tutorials, we train a simple neural network to perform a simple numerical operation.

First Assignment

In today's lesson, we will implement different functions. The first one is a simple TensorFlow-PyTorch script to find the best assignment to minimize a two-variable function. In particular, the file to complete is:

```
RL-Lab/lessons/lesson_6_code.py
```

Inside the file, a python class and a function are partially implemented. The objective of the first assignment is to complete it.

- **def find_minimum_keras()**
- **def find_minimum_torch()**

Expected results can be found in:

```
RL-Lab/results/lesson_6_results.txt
```

Non-Linear Optimization

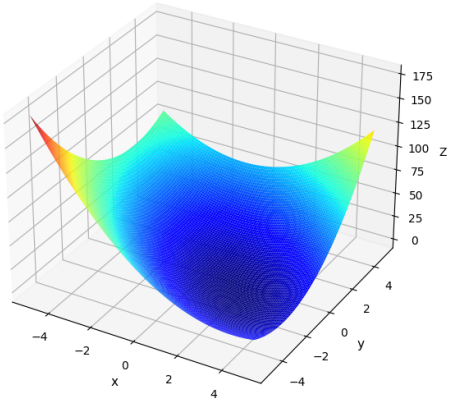

Figure: Visualization of the objective function to minimize $2x^2 + 2xy + 2y^2 - 6x$

- This function has a global minimum in -6, obtained assigning $x = 2$ and $y = -1$. The objective of the first assignment is to exploit TensorFlow to find these values.
- Remember to consult the tutorial (here) for hints and suggestions on how to solve the problem.

Second Assignment

The second assignment consists of training a neural network to predict the reward of an input state. The assignment is subdivided into three stages, corresponding to three functions to implement:

- **def create_DNN_keras()**
- **class TorchModel(nn.Module)**
- **def collect_random_trajectories()**
- **def train_DNN_keras()**
- **def train_DNN_torch()**

Expected results can be found in:

```
RL-Lab/results/lesson_6_results.txt
```

create_DDN_keras() and collect_random_trajectories()

1. For the first function, you should exploit the code snippet from the tutorial (here). Given the number of input, output, layers, and sizes, the function returns a neural network of the desired shape.
2. The second function, collect_random_trajectories, shows how to exploit the interactions with the environment to collect a dataset (the basic structure of reinforcement learning). You should implement a function that returns a two-dimensional array with the information of each episode, with the following structure:

```
memory_buffer = []
memory_buffer.append( [state, action, next_state, reward, done] )
```

Exploration Policy

To collect the data from the interaction with the environment, one can use different policies. In this lesson, you can use a random policy. For the implementation, you can take inspiration from lesson 4.

create_DDN_torch()

- Similarly, for this function, you should exploit the code snippet from the PyTorch tutorial (here). Given the number of input, output, layers, and sizes, the function returns a neural network of the desired shape.
- For simplicity, we provide the code to initialize the Keras and Torch models with the same weights and biases. Hence, the results should be the same for both models.

train_DDN_keras() and train_DDN_torch()

For these functions, you should exploit the code snippet from the Tensorflow-PyTorch tutorials (here, *the function requires a memory_buffer to perform the training*). The resulting DNN should be able to predict the reward from a given input state. From the given results, you may notice two main problems:

Wrong Prediction of the Goal State

The trained network correctly predicts the initial state (i.e., 0) but fails the estimation of the goal state (i.e., 48). The reason is that the agent rarely sees the goal state and there is not a lot of data about it.

Meaning of the Prediction

Even assuming the agent is able to predict all the rewards correctly, this information is not enough to build a policy. The DNN should predict the value (or expected reward) for choosing an action and not only the immediate reward.

In the next lesson, we will implement Deep Q-Network (DQN) to solve these problems.